OpenTelemetry in the RAG Chatbot

What this guide covers

This guide describes the OpenTelemetry code currently added to this repository. It focuses on the chat request trace across WebSocket and Kafka, and on metrics for retrieval, answer caching, and model/dependency calls. It explains what each helper does, where it is called, how metrics reach Prometheus, and how to inspect the results.

OpenTelemetry (OTel) is a vendor-neutral set of APIs, SDKs, and data formats for producing telemetry. Telemetry is operational data that helps answer questions such as: Which part of a request was slow? How often does the answer cache hit? Are Elasticsearch calls failing? How many tokens did Gemini report using?

Traces and metrics

Traces

A trace follows one request through multiple components. It consists of spans. A span represents an operation, has a start and end time, and can carry safe attributes and an error status. Parent-child relationships show the order and nesting of work.

In this application the WebSocket handler starts a span for a message, adds W3C trace context to the Kafka request, and the retrieval router extracts that context and continues the trace. The router then creates spans for processing and the retrieval workflow. It propagates the context on the Kafka response, and the WebSocket response consumer continues it. The Collector sends these traces to Jaeger.

Metrics

Metrics aggregate measurements over time. A counter increases, a histogram records a distribution (for example latency), and a gauge represents a current value. This implementation uses counters and histograms. Prometheus scrapes the Collector's Prometheus exporter and retains the time series for queries and charts.

Traces are useful for diagnosing an individual request. Metrics are useful for spotting patterns across many requests. For example, a trace can show why one query took two seconds; a latency histogram can show whether retrieval is getting slower overall.

End-to-end architecture

```
WebSocket message
  -> chat.websocket.message span
  -> Kafka chat-requests (traceparent/tracestate headers)
  -> retrieval-router extracts context
  -> chat.request.process span
     -> retrieval.workflow span
        -> Redis answer-cache lookup metrics
        -> Gemini intent/refinement/synthesis dependency metrics
        -> Ollama query-embedding dependency metrics
        -> Elasticsearch BM25 and ANN dependency metrics
        -> chunk-count and relevance/fallback metrics
     -> cache write metrics (when answer is cacheable)
  -> Kafka chat-responses (trace context headers)
  -> WebSocket response span

Application OTel SDK -> OTLP/gRPC -> OpenTelemetry Collector
  -> Jaeger (traces)
  -> Prometheus exporter :8889 -> Prometheus scrape -> PromQL/UI :9090
```

Trace context is propagated with the W3C Trace Context format in Kafka headers. The metrics described below use low-cardinality attributes such as provider, operation, outcome, stage, and token type. Request text, prompts, answers, identities, roles, and documents are not metric labels.

Telemetry setup: `app/telemetry.py`

`configure_telemetry(service_name)`

This initializes the OpenTelemetry SDK once in a process.

1. It reads `OTEL_EXPORTER_OTLP_ENDPOINT` (default `otel-collector:4317`) and creates a shared `Resource` containing `service.name` and `service.namespace`. A resource identifies which service emitted telemetry; it is not a per-request label. 2. It creates a `TracerProvider` and an OTLP gRPC span exporter, attaches a `BatchSpanProcessor`, and registers the provider globally. Batching avoids sending a network request for every span event. 3. It creates a `PeriodicExportingMetricReader` and OTLP metric exporter. The default export interval is 10 seconds and can be changed with `OTEL_METRIC_EXPORT_INTERVAL_MS`. 4. It creates and globally registers a `MeterProvider` and registers provider shutdown handlers with `atexit`.

The environment flag `OTEL_SDK_DISABLED=true` disables initialization. The endpoint's insecure setting is controlled by `OTEL_EXPORTER_OTLP_INSECURE` (true in local Compose). This configuration is intended for local development; production should use authenticated, encrypted transport and an appropriately secured Collector.

The WebSocket process and retrieval-router call this function during module initialization, with service names `websocket` and `retrieval-router`. The globally registered SDK providers back instruments created in `app/observability_metrics.py`.

`inject_trace_headers()`

Uses the configured OpenTelemetry propagator to serialize the current span context into W3C headers and returns them in the list-of-tuples shape used by the Kafka client. The `traceparent` header carries trace ID, parent span ID, and trace flags. This is context metadata, not application message content.

`extract_trace_context(headers)`

Converts Kafka headers into a carrier for the propagator and extracts the remote context. It skips null values and decodes byte values. A consumer uses the returned context when it starts a span so that the new span is connected to the original WebSocket trace.

Metric definitions and helpers: `app/observability_metrics.py`

The module obtains a meter named `e2e-langgraph-rag` and creates these instruments at import time.

Instrument • OTel type • Attributes • Meaning

`rag.cache.operations` • Counter • `operation`, `outcome` • Cache lookup/write counts: hit, miss, error, or success.

`rag.cache.operation.duration` • Histogram • `operation`, `outcome` • Time spent in an answer-cache operation, in seconds.

`rag.retrieval.operations` • Counter • `outcome` • Retrieval executions by success, low relevance, no results, or error.

`rag.retrieval.duration` • Histogram • `outcome` • End-to-end RetrievalAgent duration, in seconds.

`rag.retrieval.chunks` • Histogram • `stage` • Number of chunks at BM25, ANN, RRF, reranked, and returned stages.

`rag.retrieval.relevance` • Histogram • none • Distribution of the top retrieval relevance score, clamped to 0..1.

`rag.retrieval.fallbacks` • Counter • `reason` • Count of retrieval fallbacks, currently low relevance or retrieval error.

`rag.dependency.operations` • Counter • `provider`, `operation`, `outcome` • Dependency/model call count by final result.

`rag.dependency.duration` • Histogram • `provider`, `operation`, `outcome` • Duration of a dependency/model call, in seconds.

`rag.model.tokens` • Counter • `provider`, `operation`, `token_type` • Provider-reported Gemini input/output token usage.

Prometheus translates OTel metric names by replacing dots with underscores. Depending on instrument type, it also adds Prometheus suffixes such as `_total`, `_bucket`, `_sum`, and `_count`.

`error_outcome(exc)`

Maps an exception to one of the finite outcomes `timeout`, `rate_limited`, or `error`. It checks timeout class/name, HTTP status 429, and common rate-limit/resource-exhausted names. This keeps labels bounded and makes failures easy to group. It does not classify every provider-specific exception perfectly; unrecognized failures become `error`.

`record_dependency(provider, operation, outcome, duration_seconds)`

Adds one to `rag.dependency.operations` and records the elapsed seconds in `rag.dependency.duration`, using the same three attributes for both instruments. It is the shared low-level recorder used by the context manager below.

`dependency_operation(provider, operation)`

A context manager to wrap a dependency call. It starts a monotonic timer, records `success` if the block returns, or records a mapped failure outcome and re-raises the original exception. Re-raising preserves the existing application's error handling and fallback behavior. Example:

```
with dependency_operation("elasticsearch", "bm25_search"):
    response = es.search(...)
```

It measures the operation duration without storing the query or response in metric attributes.

`record_cache(operation, outcome, duration_seconds)`

Records the cache counter and duration histogram. The cache utility calls it once in a `finally` block, so both normal results and handled errors are counted.

`record_retrieval(outcome, duration_seconds, relevance_score=None)`

Records retrieval count and duration, and optionally records the top result score. The score is clamped to the interval 0..1 so one unexpected value cannot distort the histogram.

`record_chunks(stage, count)`

Records a nonnegative chunk count with a stage label. The stage is a fixed code-defined value, not user input. Histograms make it possible to inspect distributions and percentiles (subject to Prometheus histogram configuration).

`record_fallback(reason)`

Increments the fallback counter using a finite reason label. Callers should only use known constant strings; arbitrary text would create unbounded time series and could leak sensitive data.

`record_model_tokens(operation, token_type, count)`

Adds positive token counts to the model token counter with provider fixed to `gemini`, and a bounded operation and token type. It ignores zero or negative values. Token counts are usage quantities, not a cost estimate.

Where instrumentation is applied

Answer cache: `app/utils/answer_cache.py`

`get_cached_answer(query, memory_scope_id)`

The cache-key logic remains responsible for scoping a cached answer to the query and authorization/memory scope. When a key is available, the function times the Redis `get` call inside `dependency_operation("redis", "answer_cache_get")`. It records cache outcome `hit` when a value is returned, `miss` when Redis returns no value, and `error` when Redis access or decoding fails. Cache errors are logged and treated as a miss so the chat can continue through retrieval.

The query and scope are not attached to telemetry. The cache key itself is not recorded.

`cache_answer(query, memory_scope_id, result)`

Times the Redis `set` call, records Redis dependency success/error, and records cache write outcome and duration in `finally`. Cache write problems are logged and do not fail an otherwise successful answer. The existing TTL and permission-scoped cache-key logic are separate from OpenTelemetry.

BM25 retrieval: `app/utils/text_bm25.py`

`BM25Retriever.search(...)`

Wraps the Elasticsearch search request in `dependency_operation("elasticsearch", "bm25_search")`. Elasticsearch call duration and outcome therefore feed dependency metrics. After filtering hits to usable content, it records the number of BM25 candidates with `record_chunks("bm25_candidates", count)`. Empty queries or missing allowed roles return before making an Elasticsearch request and therefore do not create a dependency-call observation.

Hybrid retrieval and reranking: `app/utils/chunk_retriever.py`

`ChunkRetriever._ann(...)`

Wraps the Elasticsearch k-nearest-neighbor request as `elasticsearch/ann_search`. After decoding usable hits, it records `ann_candidates`.

`ChunkRetriever.retrieve(...)`

After BM25 and ANN retrieval, it merges candidates with reciprocal rank fusion and records `rrf_candidates`. It calls the cross-encoder and records `reranked`, then selects the top `top_k` results and records `returned`.

The cross-encoder's duration is not currently separately measured. Its chunk counts are measured. BM25 and ANN are separately measured Elasticsearch dependency operations.

Retrieval orchestration: `app/agents/retrieval_agents/retrieval_agent.py`

`RetrievalAgent.execute()`

Wraps the Ollama embedding call in `dependency_operation("ollama", "query_embedding")`. It starts an end-to-end retrieval timer, calls `ChunkRetriever`, derives a bounded outcome (`success`, `low_relevance`, or `no_results`), and records duration and top relevance score. Low relevance/no-result paths are considered fallbacks and get a fallback counter. Exceptions record retrieval `error`, record `retrieval_error`, and keep the existing workflow fallback response behavior.

Empty queries and requests without allowed roles return early before the retrieval timer's normal recording path; these validation exits are not currently included in `rag.retrieval.operations`.

Gemini: `app/configs/llm_config.py`

`_record_gemini_usage(operation, response)`

Reads `usage_metadata` from a successful Gemini response and records prompt and candidate token counts as input and output. It safely skips token metrics when the response has no usage metadata. It never records prompt text, response text, model content, or cost.

`ask_gemini_structured(...)`

Wraps the Gemini structured-generation call in `dependency_operation("gemini", "structured_generation")`, recording call duration and final success/failure. On success it records usage metadata, parses the JSON response, and returns it. Existing exception fallback behavior remains in place; failures are observable even though the caller receives the existing fallback dictionary.

`fire_fast_modal_request_chat(...)`

Wraps Gemini text generation as `gemini/answer_generation`, records usage metadata on successful provider responses, and returns the trimmed text. Provider errors are measured before the existing handler returns an empty string.

Trace instrumentation by service

WebSocket service

`app/websocket_service/main.py` configures the `websocket` telemetry resource. The `chat.websocket.message` span covers message validation/signing and Kafka request production. The trace context is injected into the Kafka headers. No message body or JWT is added as a span attribute.

`app/websocket_service/kafka_consumer.py` extracts trace context from response message headers and starts `chat.websocket.response`; parse/delivery errors mark the span as an error.

Retrieval router

`app/router_service/request_response_router.py` configures the `retrieval-router` service. `_handle_request` extracts the request context and starts `chat.request.process`. `process_request` starts `retrieval.workflow` around the LangGraph call. The response Kafka message gets injected context so the WebSocket response consumer can continue the trace.

The router also performs memory operations after workflow completion. Those operations are not all independently wrapped in spans or dependency metrics by this instrumentation.

Collector and Prometheus setup

`observability/otel-collector-config.yaml` configures an OTLP/gRPC receiver on port 4317, a memory limiter and batch processor, a Jaeger OTLP exporter for traces, and a Prometheus exporter listening on port 8889 for metrics.

`observability/prometheus.yml` tells Prometheus to scrape `otel-collector:8889` every 15 seconds. Prometheus is exposed at host port 9090. Jaeger is exposed at host port 16686.

The `docker-compose.yml` additions run the Collector, Jaeger, and Prometheus. The app services receive the OTLP endpoint through environment variables. Compose starts Redis and Elasticsearch dependencies; Ollama must be reachable at the configured `OLLAMA_URL` (defaulting to `host.docker.internal:11434`).

Run and inspect

Start the app and local telemetry stack from the repository root:

```
docker compose up -d --build websocket retrieval-router prometheus
```

Send chat requests so the code paths emit telemetry. The SDK exports every 10 seconds by default and Prometheus scrapes every 15 seconds, so allow several seconds for the first values to appear.

Open:

- Prometheus: `http://localhost:9090`
- Jaeger: `http://localhost:16686`

In Jaeger, select `websocket` or `retrieval-router` and find a recent trace. In Prometheus, use the query box and metric autocomplete (`rag_`) to see exported names.

Cache hit ratio

```
sum(rate(rag_cache_operations_total{operation="lookup",outcome="hit"}[5m]))
/
sum(rate(rag_cache_operations_total{operation="lookup",outcome=~"hit|miss"}[5m]))
```

This excludes cache errors from the denominator. If there are no lookups in the selected period, Prometheus may show no value rather than zero.

Dependency failures per second

```
sum by (provider, operation, outcome) (
  rate(rag_dependency_operations_total{outcome=~"error|timeout|rate_limited"}[5m])
)
```

P95 dependency duration

```
histogram_quantile(
  0.95,
  sum by (le, provider, operation) (
    rate(rag_dependency_duration_bucket[5m])
  )
)
```

Gemini token rate

```
sum by (operation, token_type) (rate(rag_model_tokens_total[5m]))
```

What this implementation does not measure yet

- It does not run periodic active health checks. Dependency health is inferred from calls that happen; an idle provider creates no fresh evidence of availability.
- It does not count SDK-internal retries. The dependency metric reports the final outcome seen by this code, not every attempt.
- It does not estimate money/cost. Prices vary by model, region, modality, and date; token counts alone are not a reliable cost figure.
- Token metrics depend on Gemini returning usage metadata. Ollama embedding token counts are not recorded.
- It does not separately measure reranker latency or every memory/ingestion operation.

- Prometheus and Jaeger here are local development services. Jaeger uses in-memory storage; telemetry retention and security should be designed for production.

Adding instrumentation to another operation

1. Choose a stable provider and operation name such as `redis/session_read`. 2. Wrap the actual call with `dependency_operation(provider, operation)` to record elapsed time and success/final failure. 3. Add a domain-specific counter or histogram only when it answers a useful operational question; use `record_chunks`, `record_cache`, or the underlying instrument pattern as appropriate. 4. Use only finite, low-cardinality attributes. Never use raw request text, user/session IDs, document names, or arbitrary exception messages as labels. 5. Add the expected measurement and a PromQL query to the observability guide, and check that the service configures telemetry before emitting measurements.

Glossary

- **Instrument:** An API object used by application code to produce a metric.
- **Counter:** A cumulative count that increases; queries commonly use `rate()` over a time window.
- **Histogram:** A distribution of measurements, usually queried for buckets or percentiles.
- **Meter:** The named OpenTelemetry API used to create metric instruments.
- **Provider:** SDK configuration that supplies tracers/meters and exporters.
- **Resource:** Identifies the emitting service and deployment attributes.
- **Exporter:** Sends SDK telemetry to another system, here using OTLP/gRPC to the Collector.
- **Collector:** Receives, processes, and exports telemetry independently of application code.
- **Cardinality:** Number of distinct time series created by combinations of metric names and attributes. Unbounded labels can overwhelm metrics storage and expose sensitive values.
- **Trace context:** IDs and flags propagated across process boundaries to connect spans into a distributed trace.